

C0_5 AT_2 ASSIGNMENT 1

DSA0103 – C++ PROGRAMMING

NAME: Ranjitha. R

REG NO: 192421285

1. Develop a Banking Management System that supports Savings, Current, and Loan Accounts. The system must allow users to perform operations such as account creation, deposit, withdrawal, balance enquiry, and loan processing.(CO5)

Requirements:

1. Define an abstract base class Account with virtual and pure virtual functions for common operations (deposit, withdraw, displayBalance, getAccountType).
2. Use runtime polymorphism (base class pointers/references) so the correct accountspecific behavior executes based on the actual object type.
3. Handle errors using exception handling — insufficient balance, invalid account number, negative deposit, and invalid withdrawal.
4. Demonstrate throwing, catching, and rethrowing exceptions during a transaction (e.g., a failed fund transfer).
5. Use exception specifications (throw(...)) on functions to declare the exceptions they may raise.

1. Banking Management System

C++ Program

```
#include <iostream>

#include <string>

using namespace std;

class Account {
protected:
    int accountNo;
    double balance;
public:
    Account(int no, double b) {
        accountNo = no;
        balance = b;
    }
    virtual void deposit(double amount) = 0;
    virtual void withdraw(double amount) = 0;
    virtual void displayBalance() = 0;
    virtual string getAccountType() = 0;
    virtual ~Account() {}
```

```

};

class SavingsAccount : public Account {
public:
    SavingsAccount(int no, double b) : Account(no, b) {}
    void deposit(double amount) {
        if (amount <= 0)
            throw "Invalid deposit amount";
        balance += amount;
    }
    void withdraw(double amount) {
        if (amount <= 0)
            throw "Invalid withdrawal amount";
        if (amount > balance)
            throw "Insufficient balance";
        balance -= amount;
    }
    void displayBalance() {
        cout << "Savings Account Balance: " << balance << endl;
    }
    string getAccountType() {
        return "Savings";
    }
};

class CurrentAccount : public Account {
public:
    CurrentAccount(int no, double b) : Account(no, b) {}
    void deposit(double amount) {
        if (amount <= 0)
            throw "Invalid deposit amount";
        balance += amount;
    }

```

```

    }

    void withdraw(double amount) {
        if (amount <= 0)
            throw "Invalid withdrawal amount";
        if (amount > balance)
            throw "Insufficient balance";
        balance -= amount;
    }

    void displayBalance() {
        cout << "Current Account Balance: " << balance << endl;
    }

    string getAccountType() {
        return "Current";
    }
};

class LoanAccount : public Account {
public:
    LoanAccount(int no, double b) : Account(no, b) {}

    void deposit(double amount) {
        if (amount <= 0)
            throw "Invalid loan payment";
        balance -= amount;
        if (balance < 0)
            balance = 0;
    }

    void withdraw(double amount) {
        throw "Withdrawal not allowed from Loan Account";
    }

    void displayBalance() {
        cout << "Loan Amount Remaining: " << balance << endl;
    }
};

```

```

    }

    string getAccountType() {
        return "Loan";
    }
};

int main() {
    try {
        Account *acc;
        SavingsAccount savings(101, 5000);
        acc = &savings;
        cout << "Account Type: "
             << acc->getAccountType() << endl;
        acc->deposit(2000);
        acc->withdraw(1000);
        acc->displayBalance();
        cout << "\nTesting invalid withdrawal..." << endl;
        acc->withdraw(10000);
    }
    catch (const char *msg) {
        cout << "Error: " << msg << endl;
        // Rethrowing exception
        try {
            throw;
        }
        catch (const char *msg2) {
            cout << "Exception rethrown and caught again: "
                 << msg2 << endl;
        }
    }

    cout << "\nLoan Account Example" << endl;
}

```

```

try {
    LoanAccount loan(201, 10000);

    Account *acc = &loan;

    acc->displayBalance();

    acc->deposit(3000);

    acc->displayBalance();

    acc->withdraw(1000);

}
catch (const char *msg) {
    cout << "Error: " << msg << endl;
}

return 0;
}

```

Sample Output

The screenshot shows a C++ IDE with a code editor on the left and an output window on the right. The code in the editor is a C++ program that demonstrates exception handling with a try-catch block. The output window shows the results of the program's execution, including account balances, error messages, and a successful execution status.

```

main.cpp Run
85 public:
86     string getAccountType() {
87     }
88 };
89
90
91 int main() {
92     try {
93         Account *acc;
94
95         SavingsAccount savings(101, 5000);
96
97         acc = &savings;
98
99         cout << "Account Type: "
100             << acc->getAccountType() << endl;
101
102         acc->deposit(2000);
103         acc->withdraw(1000);
104         acc->displayBalance();
105
106         cout << "\nTesting invalid withdrawal..." << endl;
107
108         acc->withdraw(10000);
109     }
110     catch (const char *msg) {
111         cout << "Error: " << msg << endl;
112
113         // Rethrowing exception
114         try {
115             throw;
116         }
117         catch (const char *msg2) {
118             cout << "Exception rethrown and caught again: "
119                 << msg2 << endl;
120         }
121     }
122
123     cout << "\nLoan Account Example" << endl;
124
125     try {
126         LoanAccount loan(201, 10000);
127
128         Account *acc = &loan;
129
130         acc->displayBalance();
131         acc->deposit(3000);
132         acc->displayBalance();
133
134         acc->withdraw(1000);
135     }
136     catch (const char *msg) {
137         cout << "Error: " << msg << endl;
138     }
139 }

```

Output

```

Account Type: Savings
Savings Account Balance: 6000

Testing invalid withdrawal...
ERROR!
Error: Insufficient balance
Exception rethrown and caught again: Insufficient balance

Loan Account Example
Loan Amount Remaining: 10000
Loan Amount Remaining: 7000
Error: Withdrawal not allowed from Loan Account

=== Code Execution Successful ===

```

Concepts Used

Requirement	Implementation
Abstract class	Account
Pure virtual functions	deposit(), withdraw(), etc.
Inheritance	Savings, Current, Loan
Runtime polymorphism	Account *acc
Exception handling	try-catch
Throwing	throw "..."
Rethrowing	throw;
Insufficient balance	Exception
Invalid deposit	Exception
Invalid withdrawal	Exception

2. Develop an Online Payment System for an e-commerce application that supports Credit Card, Debit Card, UPI, and Net Banking payments. The system should process different payment methods and handle transaction failures safely. (CO5)

Requirements: 1. Define an abstract base class PaymentMethod with virtual and pure virtual functions forming a common payment interface (validateDetails, pay, getMethodName).

2. Use runtime polymorphism to dynamically select and execute the correct payment method's logic at runtime.

3. Handle errors using exception handling — invalid card details, insufficient funds, incorrect UPI PIN, and failed transactions.

4. Demonstrate throwing, catching, and rethrowing exceptions during payment processing (e.g., in a payment gateway's checkout flow).

5. Use exception specifications (throw(...)) to declare possible payment-related exceptions on each function.

6. Display appropriate success or error messages based on the transaction outcome.

2. Online Payment System

C++ Program

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
class PaymentMethod {
```

```
public:
```

```
    virtual void validateDetails() = 0;
```

```

virtual void pay(double amount) = 0;
virtual string getMethodName() = 0;

virtual ~PaymentMethod() {}
};

class CreditCard : public PaymentMethod {
public:
    void validateDetails() {
        cout << "Credit Card details validated." << endl;
    }
    void pay(double amount) {
        if (amount <= 0)
            throw "Invalid payment amount";
        if (amount > 50000)
            throw "Insufficient credit limit";
        cout << "Payment of Rs." << amount
            << " made using Credit Card." << endl;
    }
    string getMethodName() {
        return "Credit Card";
    }
};

class DebitCard : public PaymentMethod {
public:
    void validateDetails() {
        cout << "Debit Card details validated." << endl;
    }
    void pay(double amount) {
        if (amount <= 0)
            throw "Invalid payment amount";

```

```

        if (amount > 20000)
            throw "Insufficient bank balance";
        cout << "Payment of Rs." << amount
            << " made using Debit Card." << endl;
    }
    string getMethodName() {
        return "Debit Card";
    }
};

class UPI : public PaymentMethod {
public:
    void validateDetails() {
        cout << "UPI ID validated." << endl;
    }
    void pay(double amount) {
        if (amount <= 0)
            throw "Invalid payment amount";
        if (amount > 10000)
            throw "UPI transaction limit exceeded";

        cout << "Payment of Rs." << amount
            << " made using UPI." << endl;
    }
    string getMethodName() {
        return "UPI";
    }
};

class NetBanking : public PaymentMethod {
public:
    void validateDetails() {

```



```

        cout << "Net Banking details validated." << endl;
    }
    void pay(double amount) {
        if (amount <= 0)
            throw "Invalid payment amount";

        if (amount > 100000)
            throw "Transaction limit exceeded";
        cout << "Payment of Rs." << amount
            << " made using Net Banking." << endl;
    }
    string getMethodName() {
        return "Net Banking";
    }
};

int main() {
    PaymentMethod *payment;
    try {
        UPI upi;
        payment = &upi;
        cout << "Payment Method: "
            << payment->getMethodName() << endl;
        payment->validateDetails();
        payment->pay(500);
        cout << "Transaction successful." << endl;
    }
    catch (const char *msg) {
        cout << "Payment Failed: " << msg << endl;
        // Rethrow exception
        try {

```

```

        throw;
    }
    catch (const char *msg2) {
        cout << "Exception rethrown: "
            << msg2 << endl;
    }
}
cout << "\nTesting another payment method" << endl;
try {
    DebitCard debit;
    payment = &debit;
    cout << "Payment Method: "
        << payment->getMethodName() << endl;
    payment->validateDetails();
    payment->pay(25000);
}
catch (const char *msg) {
    cout << "Payment Failed: " << msg << endl;
}
return 0;
}

```

Sample Output

```

main.cpp
1 public:
2     string getMethodName() const {
3         return name;
4     }
5 };
6
7 int main() {
8     PaymentMethod *payment;
9
10    try {
11        UPI upi;
12        payment = &upi;
13
14        cout << "Payment Method: "
15              << payment->getMethodName() << endl;
16
17        payment->validateDetails();
18        payment->pay(500);
19
20        cout << "Transaction successful." << endl;
21    }
22    catch (const char *msg) {
23        cout << "Payment Failed: " << msg << endl;
24
25        // Rethrow exception
26        try {
27            throw;
28        }
29        catch (const char *msg2) {
30            cout << "Exception rethrown: "
31                  << msg2 << endl;
32        }
33    }
34
35    cout << "\nTesting another payment method" << endl;
36
37    try {
38        DebitCard debit;
39        payment = &debit;
40
41        cout << "Payment Method: "
42              << payment->getMethodName() << endl;
43
44        payment->validateDetails();
45        payment->pay(25000);
46    }
47    catch (const char *msg) {
48        cout << "Payment Failed: " << msg << endl;
49    }
50
51    return 0;
52 }

```

Output

```

Payment Method: UPI
UPI ID validated.
Payment of Rs.500 made using UPI.
Transaction successful.

Testing another payment method
Payment Method: Debit Card
Debit Card details validated.
Payment Failed: Insufficient bank balance

=== Code Execution Successful ===

```

Concepts

Requirement	Implementation
Abstract class	PaymentMethod
Pure virtual functions	validateDetails(), pay()
Inheritance	CreditCard, DebitCard, UPI, NetBanking
Runtime polymorphism	PaymentMethod *payment
Exception handling	try-catch
Throwing exceptions	throw "..."
Rethrowing	throw;
Invalid amount	Exception
Failed transaction	Exception
Different payment logic	Overridden pay()